MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Bulimar Rodion FAF-242

Report

Laboratory Work 2
Cryptography

Checked by:
Zaica Maia, *university assistant*
FCIM, UTM

Chișinău – 2026

1 Purpose

The purpose of this laboratory work is to examine a monoalphabetic substitution ciphertext. The program counts the occurrences and percentages of ASCII letters in the ciphertext and applies a fixed substitution mapping to produce plaintext. The frequency table supports cryptanalysis, while the decryption mapping is supplied in the program.

2 Short Theory

A monoalphabetic substitution cipher replaces each ciphertext letter with one corresponding plaintext letter. The mapping is fixed throughout the message. In `lab_2.py`, the mapping is represented as a dictionary from ciphertext letters to plaintext letters.

For a ciphertext letter c, let $N(c)$ be its number of occurrences and let N be the total number of ASCII alphabetic characters. Its frequency is:

$$F(c) = \frac{N(c)}{N} \times 100\%.$$

Letter frequencies and repeated patterns can provide clues when analyzing substitution ciphertexts. The program calculates and displays these frequencies, but it does not automatically infer a key from them. Instead, the substitution alphabet is hardcoded after being inferred from frequency and pattern observations.

3 Program Implementation

The source file contains a long, hardcoded ciphertext in `CIPHERTEXT`. The excerpts below show the constants and functions that process it.

3.1 Substitution mapping

```
CIPHER_ALPHABET = string.ascii_lowercase
PLAIN_ALPHABET = "bqfujyncrgvkzodshwlapetixm"
SUBSTITUTION = dict(zip(CIPHER_ALPHABET, PLAIN_ALPHABET))
```

Comment: The two 26-letter alphabets are paired by position to build a dictionary. Each ciphertext letter maps to one plaintext letter. `main()` checks that `PLAIN_ALPHABET` has 26 characters and that they are all distinct before producing output.

3.2 Letter-frequency calculation

```
def letter_frequencies(text: str) -> List[Tuple[str, int, float]]:
    """Return letters ordered by frequency, with counts and percentages."""
    letters = [char.lower() for char in text if char.isalpha() and char.isascii
    ()]
    counts = Counter(letters)
    total = len(letters)
    return [
        (letter, count, count / total * 100)
        for letter, count in counts.most_common()
    ]
```

Comment: The function ignores spaces, punctuation, and non-ASCII letters; it treats uppercase and lowercase letters equally. It returns `(letter, count, percentage)` tuples ordered from most frequent to least frequent.

3.3 Substitution decryption

```python
def decrypt(text: str) -> str:
    """Decrypt while preserving capitalization, spaces, and punctuation."""
    result = []
    for char in text:
        plain = SUBSTITUTION.get(char.lower())
        if plain is None:
            result.append(char)
        elif char.isupper():
            result.append(plain.upper())
        else:
            result.append(plain)
    return "".join(result)
```

Comment: Each recognized letter is replaced using `SUBSTITUTION`. Uppercase letters remain uppercase. Characters that are not keys in the mapping, including spaces and punctuation, are copied unchanged.

3.4 Main function and output

```python
def main() -> None:
    if len(PLAIN_ALPHABET) != 26 or len(set(PLAIN_ALPHABET)) != 26:
        raise ValueError("Cheia trebuie sa fie o substitutie bijectiva a
    alfabetului.")

    print("Frecventele literelor cifrate:")
    for letter, count, percentage in letter_frequencies(CIPHERTEXT):
        print(f"{letter}: {count:4} aparitii ({percentage:5.2f}%)")

    print("\nMesajul descifrat:\n")
    plaintext = decrypt(CIPHERTEXT).replace("\u2014", "-")
    print(plaintext)
```

Comment: `main()` validates the plaintext alphabet, prints the letter-frequency table, decrypts the hardcoded ciphertext, changes em dashes to ordinary hyphens for display, and prints the resulting plaintext. The status messages are in Romanian.

4 Results

Running `python lab_2.py` completed successfully. The frequency function returned 26 distinct letters, totaling 3,658 ASCII letters. The observed frequency table was:

The decryption output begins:

> Each new setting of Alberti's disk brought into play a new cipheralphabet, in which both the plaintext...

The function preserves the characters and capitalization present in the input. The sample output also shows that spacing in the decrypted text follows the spacing in the hardcoded ciphertext; the decryption function does not restore word boundaries automatically.

Cipher letter	Count	Percentage
v	479	13.09%
w	363	9.92%
x	280	7.65%
t	279	7.63%
n	248	6.78%
p	242	6.62%
q	240	6.56%
g	226	6.18%
i	224	6.12%
s	142	3.88%
u	119	3.25%
o	114	3.12%
h	104	2.84%
z	98	2.68%
d	92	2.52%
c	88	2.41%
a	81	2.21%
f	62	1.69%
r	51	1.39%
j	51	1.39%
k	26	0.71%
l	18	0.49%
y	17	0.46%
b	6	0.16%
e	5	0.14%
m	3	0.08%



Figure 1: The frequencies

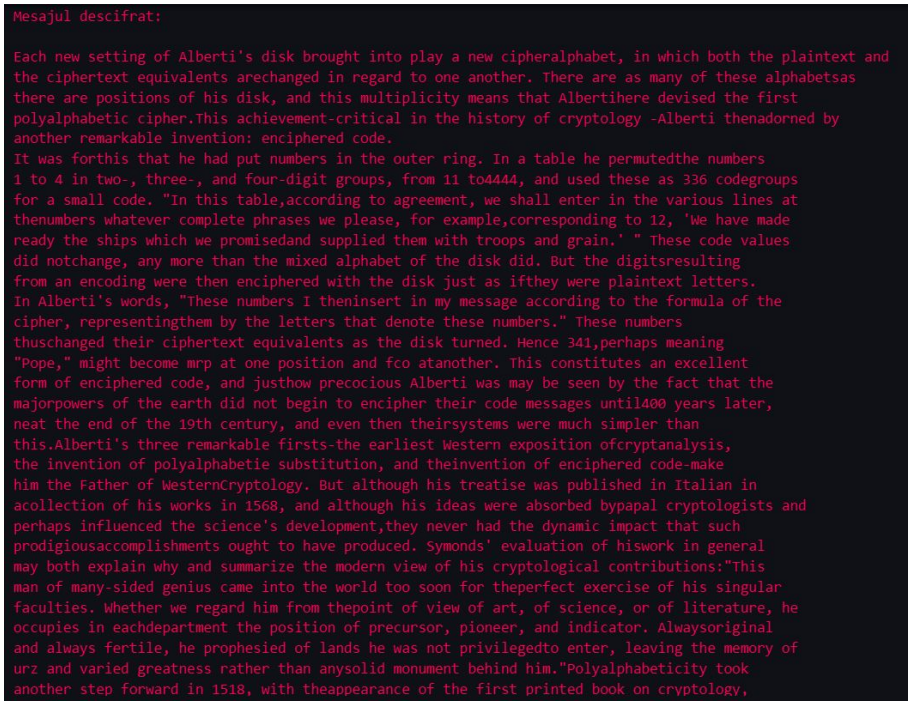


Figure 2: First part of the decrypted message

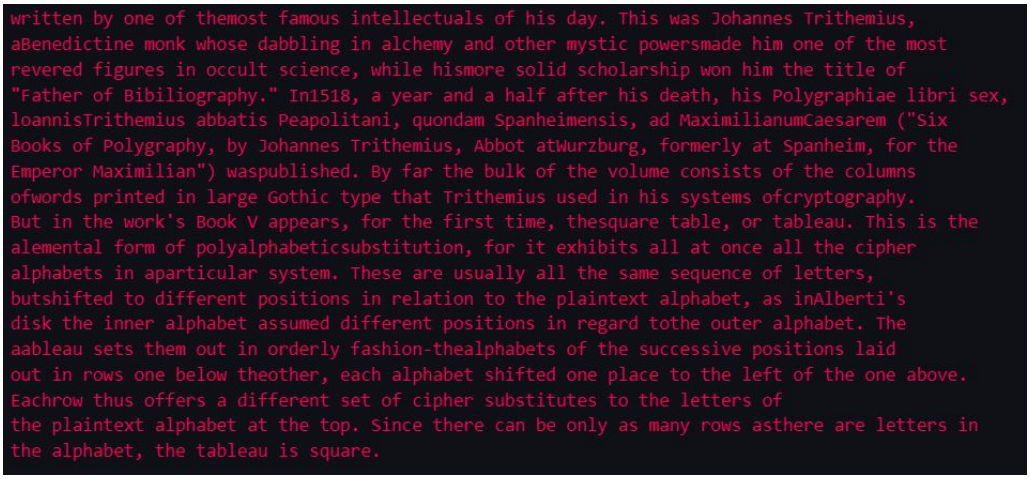


Figure 3: The second part of the decrypted message

5 Conclusions

The program demonstrates two useful cryptanalysis steps: measuring ciphertext letter frequencies and applying a substitution mapping to decrypt text. The most frequent ciphertext letter in this run is `v`, appearing 479 times (13.09%). The program's frequency analysis is descriptive rather than automatic: the mapping is already defined in `SUBSTITUTION`, so changing the ciphertext or key requires updating that mapping manually. The implementation also preserves capitalization and non-letter characters during decryption.

6 Source Code

GitHub repository: https://github.com/RodionBm/Crypto$_{lab}$2